MLOps Project

Smart Manufacturing Machines Efficiency Prediction

A production-style machine-learning system for predicting machine efficiency and enabling operational decisions through CI/CD and GitOps automation.

Python ML | Docker | Kubernetes | GitOps

Redesigned project deck

The project combines ML modeling with MLOps delivery so the model can move from notebook experimentation into a repeatable deployment workflow.

01 Manufacturing dataset

02 Predictive model

03 Containerized app

04 GitOps deployment

Business problem

Manufacturing teams need early signals of machine performance and reliability risks.

Technical goal

Build an ML app and package it for deployment using Docker and Kubernetes.

MLOps goal

Use Jenkins for CI and ArgoCD for CD, with GitHub as the source of truth.

Use Case & Dataset

The original notes identify predictive maintenance, cost management, and the Intelligent Manufacturing Dataset.

Use case focus

Predictive maintenance

Estimate machine efficiency and flag patterns that may indicate operating risk before failure.

Cost management

Support better planning by reducing downtime, waste, and reactive maintenance effort.

Industrial decision support

Convert sensor and production data into model outputs that operators can use.

Dataset used

Intelligent Manufacturing Dataset

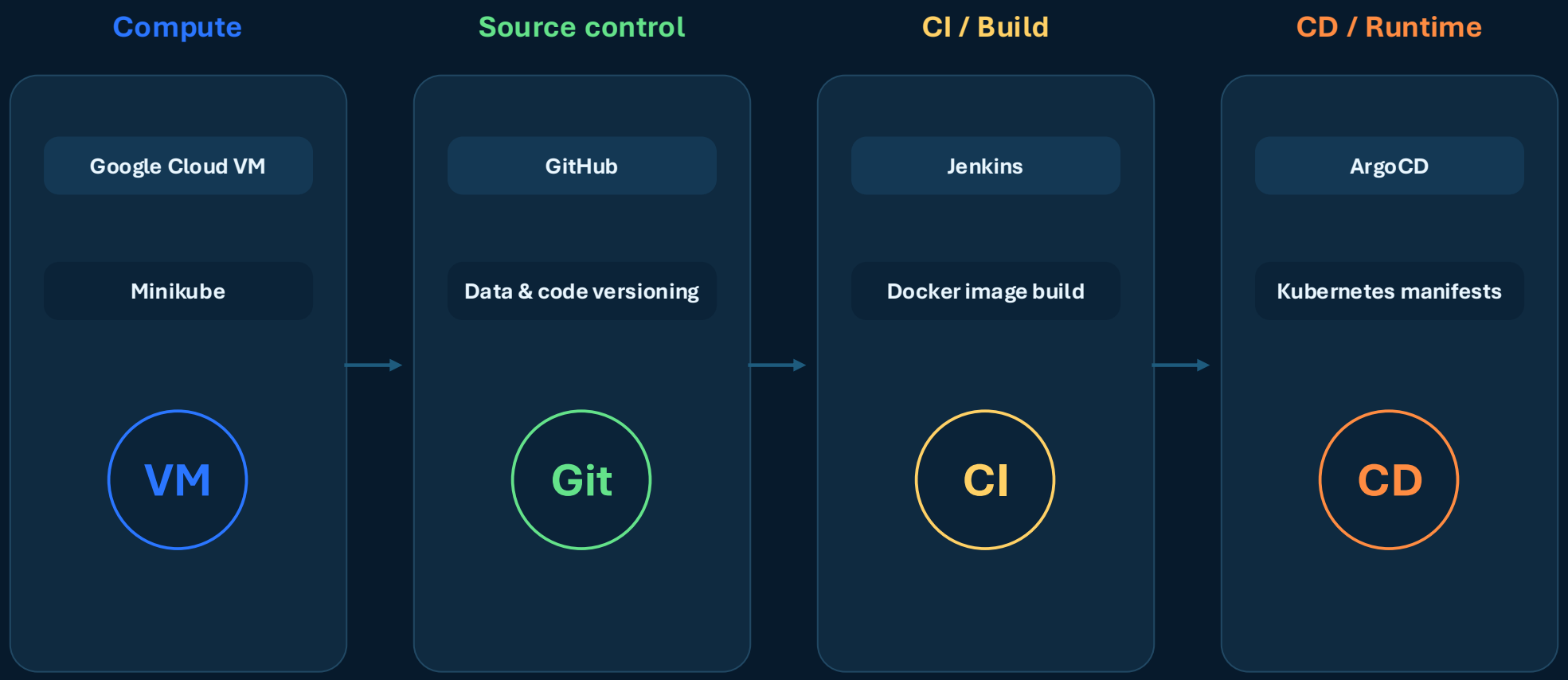
Role in the project

- Train and evaluate a machine-efficiency prediction model
- Transform raw features into clean ML-ready input
- Serve predictions through a user-facing app
- Trigger repeatable delivery through the MLOps pipeline

Data → Features → Model → App

Technology Stack

Tools from the original deck, reorganized by role in the MLOps system.



GitHub is the source of truth: code, manifests, and deployment state should flow from version-controlled changes.

01

Notebook testing

Prototype assumptions and validate data logic.

02

Data processing

Clean features and prepare train/test inputs.

03

Model training

Fit the efficiency prediction model.

04

User app

Expose predictions through an application layer.

05

Dockerfile

Package the app for consistent runtime.

Model delivery is treated as a system, not just a notebook.

The notebook validates the modeling idea, while the app, Docker image, and manifests prepare the project for repeatable deployment.

CI/CD + GitOps Flow

Jenkins builds. GitHub records the desired state. ArgoCD synchronizes Kubernetes.

1 **Developer / PC**
Push code

2 **GitHub**
Truth source

3 **Jenkins**
Build & push image

4 **Registry + Manifests**
Versioned artifact

5 **ArgoCD + K8s**
Deploy & sync app

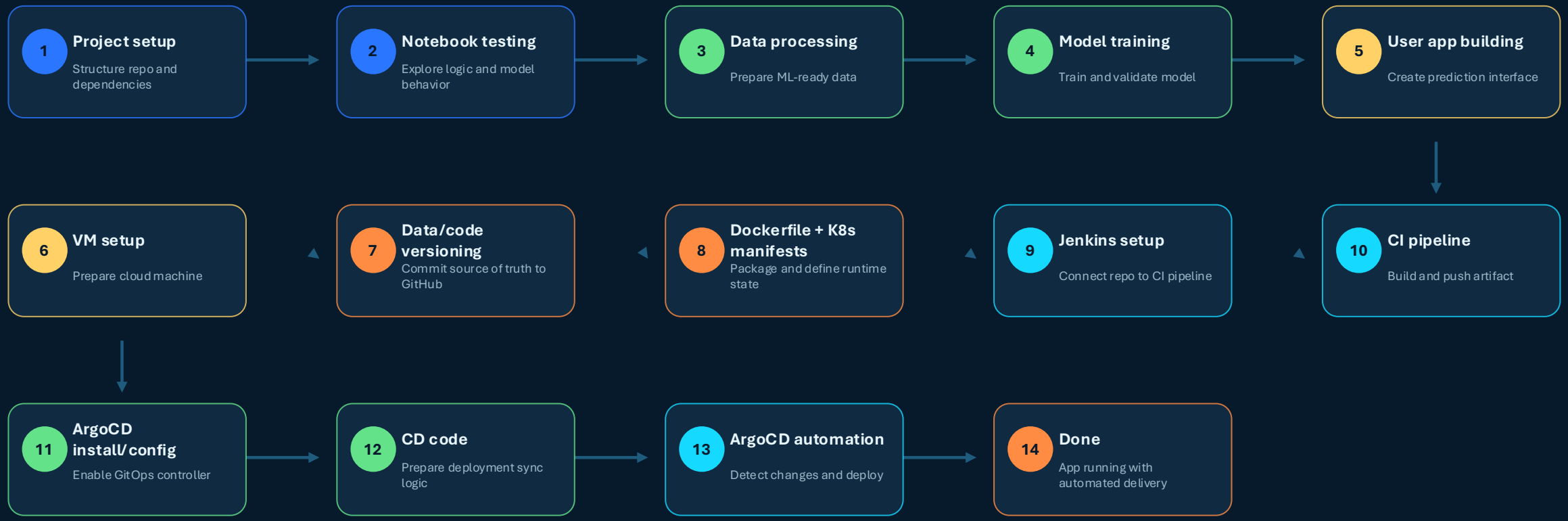
Automation behavior

A **First deployment**
Build, push, deploy, then verify the app is running.

B **Next deployments**
A GitHub change becomes the record; ArgoCD detects drift and syncs.

End-to-End Workflow Roadmap

A cleaner version of the original workflow page: setup, model, build, deploy, automate.



What This Project Demonstrates

A complete path from ML experimentation to operational deployment.

Core takeaways

ML value

Predict machine efficiency to support maintenance and cost decisions.

Engineering maturity

Move beyond notebook code into reproducible packaging and deployment.

DevOps discipline

Use CI/CD so changes are tested, built, versioned, and deployed consistently.

GitOps control

Keep deployment state traceable through GitHub and synchronized by ArgoCD.

Recommended final narrative

“This project is not only about model accuracy; it is about making the model deployable, traceable, and repeatable in a manufacturing environment.”